



CS-860: ARTIFICIAL **INTELLIGENCE**

- **Uninformed Search**
 - **UCS**
- **Informed Search**
 - **Heuristics**
 - **Greedy Search**

Text : Artificial Intelligence a Modern Approach
by Stuart Russel and Peter Norvig

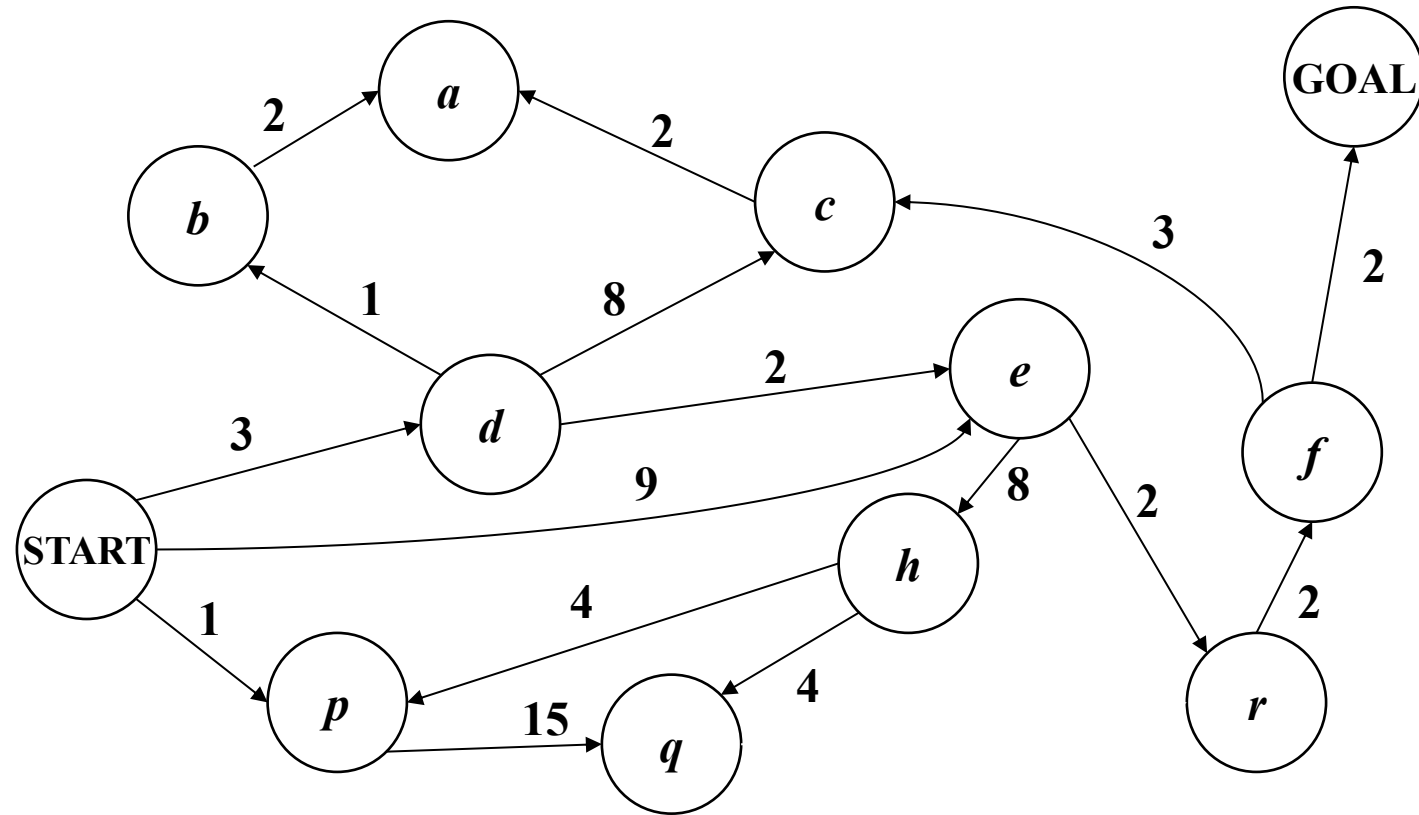
INSTRUCTOR:
DR. KUNWAR FARAZ AHMED KHAN

Uninformed Search



- **Uninformed Search**
 - Cost Sensitive Search

Cost Sensitive Search



BFS finds the shortest path in terms of number of actions.
It does not find the least-cost path. We will now cover
a similar algorithm which does find the least-cost path.



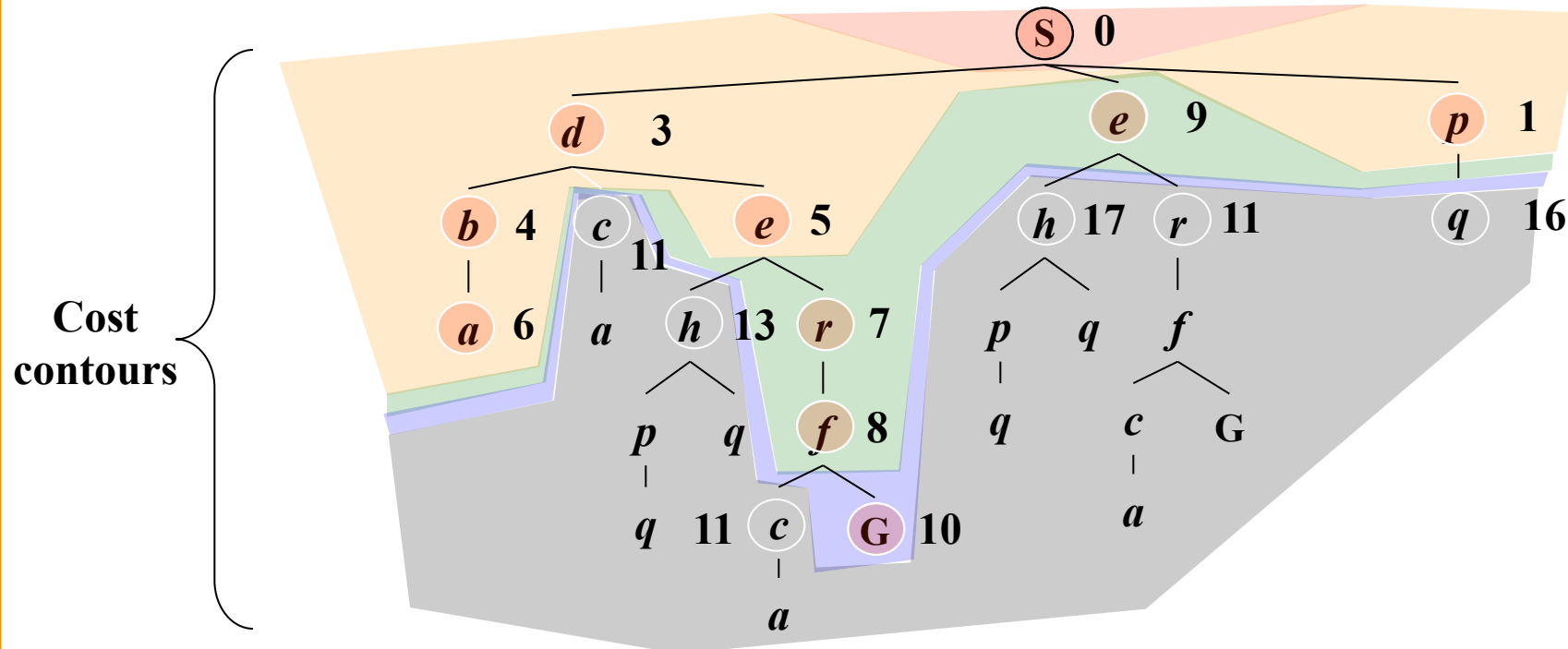
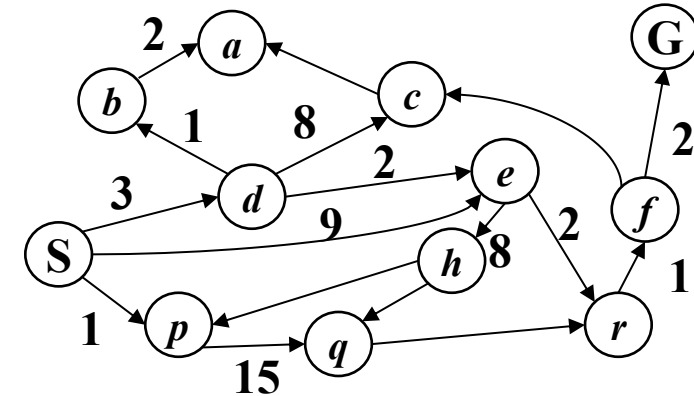
Uninformed Search



- **Uninformed Search**
 - Cost Sensitive Search
 - **Uniform Cost Search (UCS)**

Uniform Cost Search (UCS)

- *Strategy: expand a cheapest node first:*
- *Fringe is a priority queue (priority: cumulative cost; numbers in tree refer to cumulative cost)*



Uninformed Search

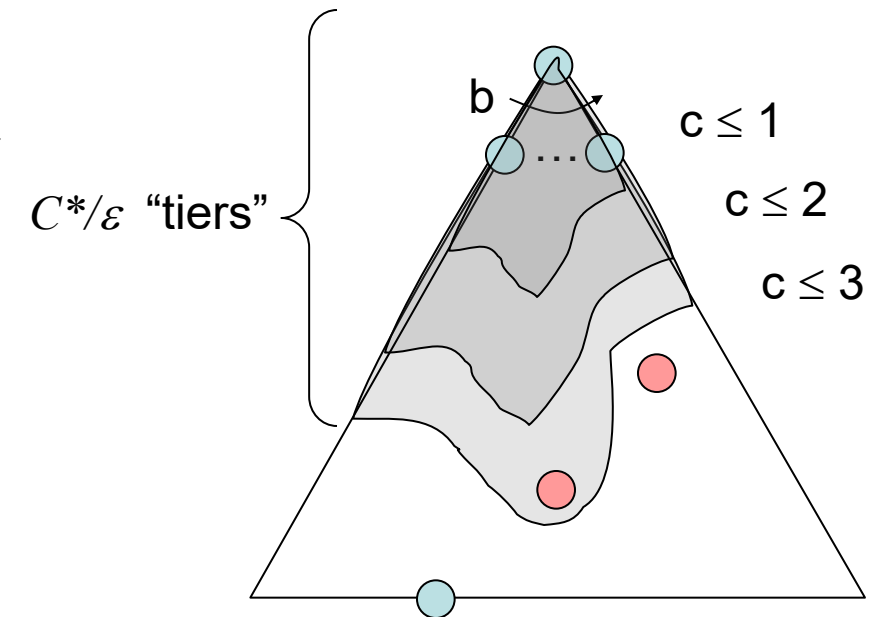


- **Uninformed Search**

- Cost Sensitive Search
- Uniform Cost Search (UCS)
- **UCS Properties**

Uniform Cost Search (UCS) Properties

- What nodes does UCS expand?
 - Processes all nodes with cost less than cheapest solution!
 - If that solution costs C^* and arcs cost at least ϵ , then the “effective depth” is roughly C^*/ϵ
 - Takes time $O(b^{C^*/\epsilon})$ (exponential in effective depth)
- How much space does the fringe take?
 - Has roughly the last tier, so $O(b^{C^*/\epsilon})$
- Is it complete?
 - Assuming best solution has a finite cost and minimum arc cost is positive, yes!
- Is it optimal?
 - Yes! (Proof next lecture via A*)



Uninformed Search

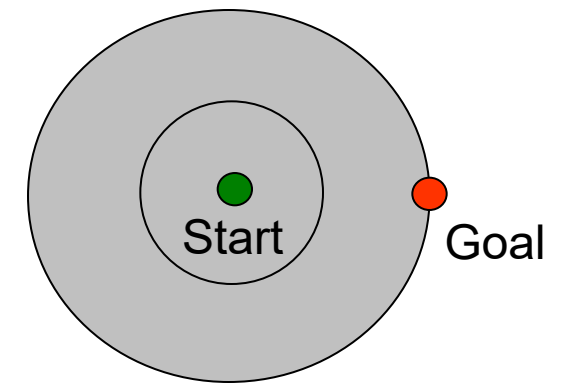
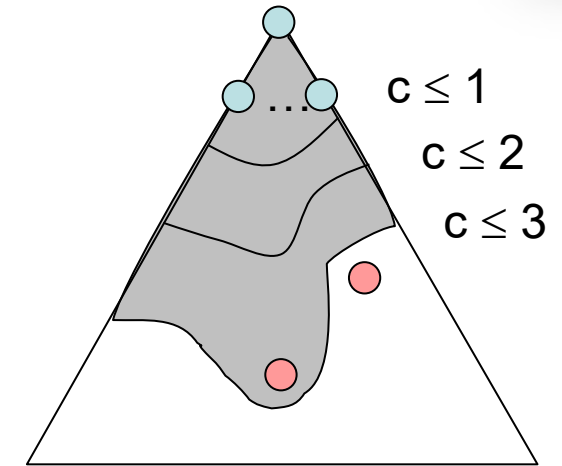


- **Uninformed Search**

- Cost Sensitive Search
- Uniform Cost Search (UCS)
- UCS Properties
- **UCS Issues**

Uniform Cost Issues

- Remember: UCS explores increasing cost contours
- The good: UCS is complete and optimal!
- The bad:
 - Explores options in every “direction”
 - No information about goal location
- We’ll fix that soon!



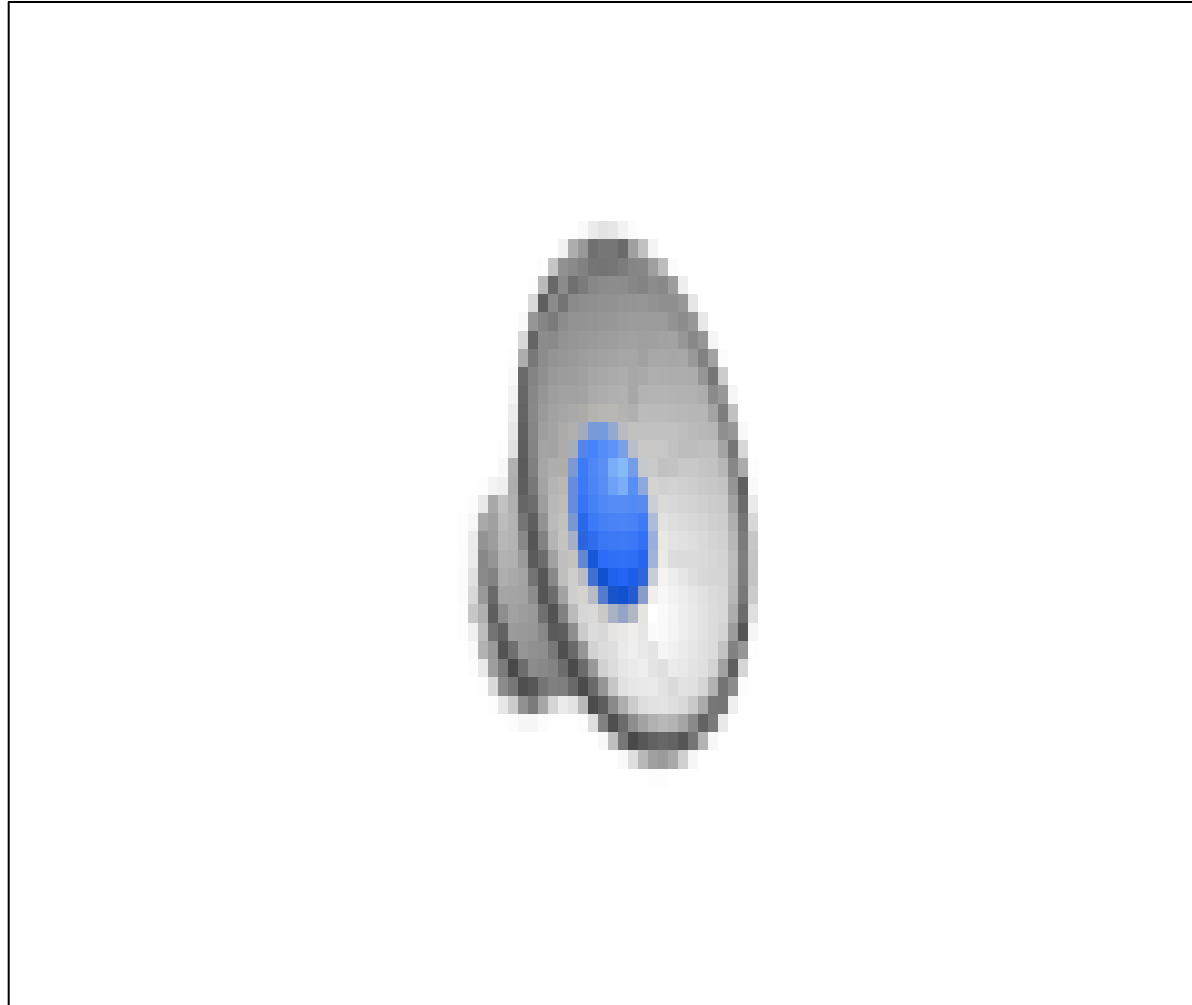
Uninformed Search



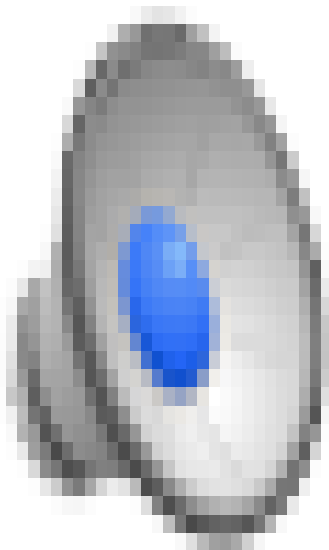
- **Uninformed Search**

- Cost Sensitive Search
- Uniform Cost Search (UCS)
- UCS Properties
- UCS Issues
- **Demos**

Demo Empty UCS



Demo Maze with Deep/Shallow Water – DFS, BFS, UCS (part I)

- 



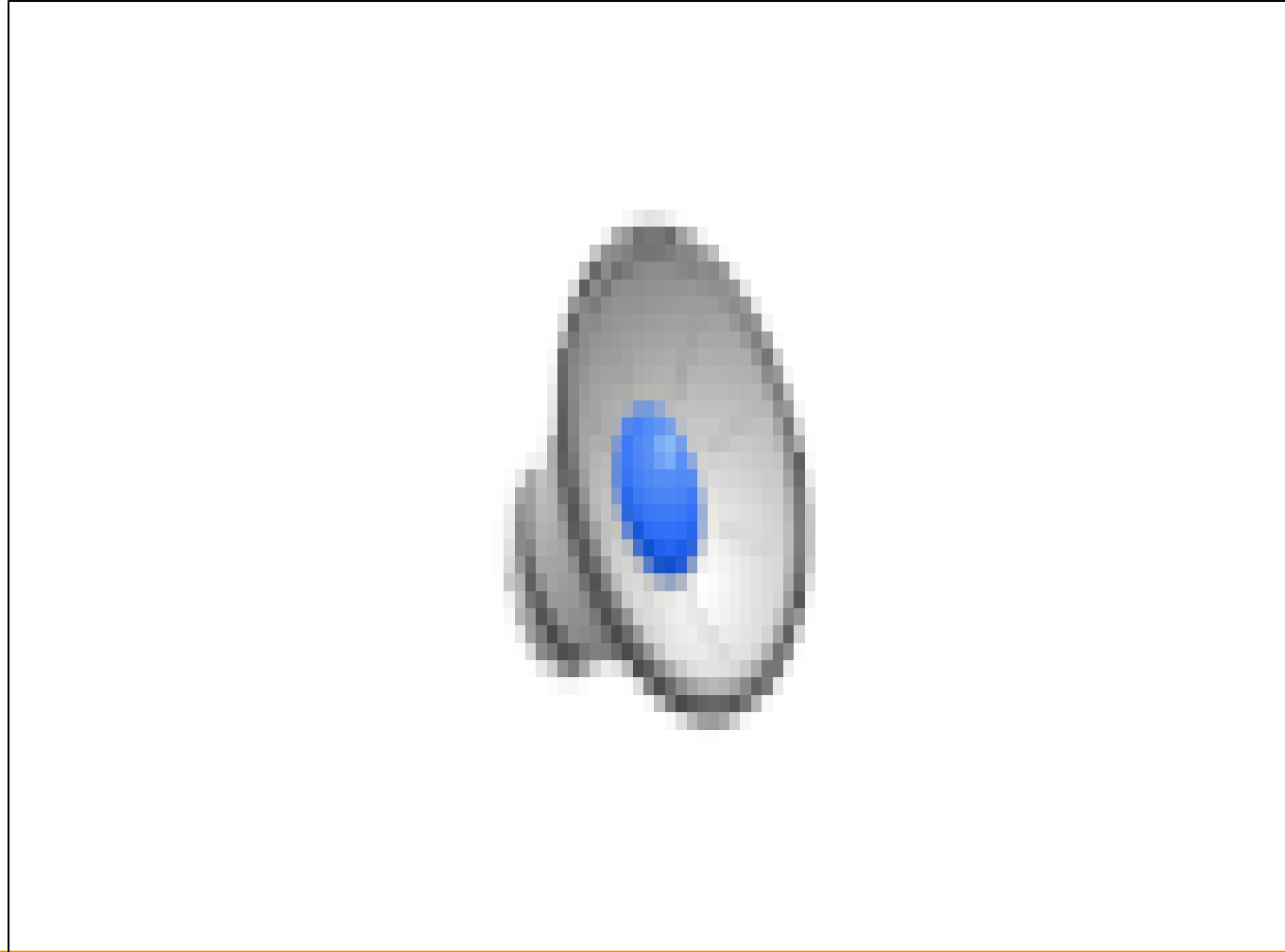
Uninformed Search



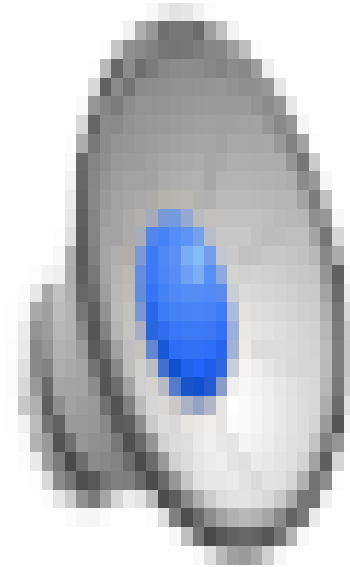
- **Uninformed Search**

- Cost Sensitive Search
- Uniform Cost Search (UCS)
- UCS Properties
- UCS Issues
- **Demos**

Demo Maze with Deep/Shallow Water – DFS, BFS, UCS (part II)



- ## Demo Maze with Deep/Shallow Water – DFS, BFS, UCS (part III)



Uninformed Search

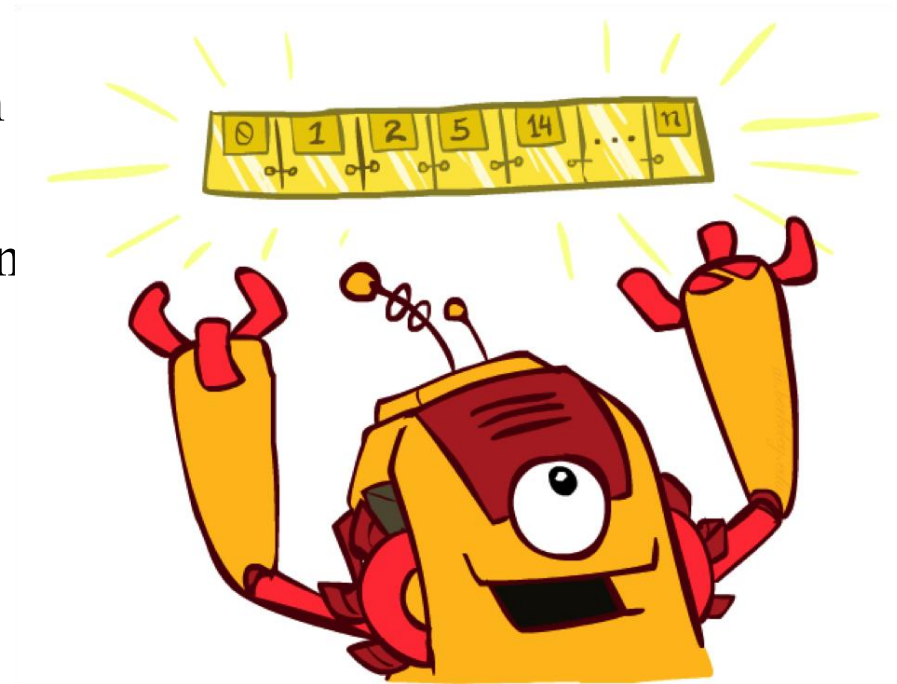


- **Uninformed Search**

- Cost Sensitive Search
- Uniform Cost Search (UCS)
- UCS Properties
- UCS Issues
- Demos
- **The One Queue**

The One Queue

- All these search algorithms are the same except for fringe strategies
 - Conceptually, all fringes are priority queues (i.e., collections of nodes with attached priorities)
 - Practically, for DFS and BFS, you can avoid the $\log(n)$ overhead from an actual priority queue, by using stacks and queues
 - Can even code one implementation that takes a variable queuing object



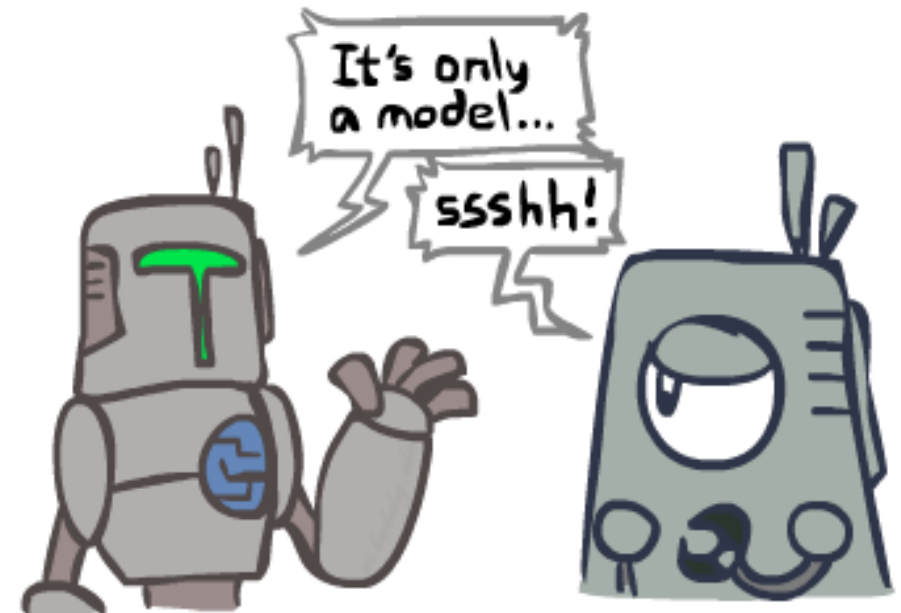
Uninformed Search



- Uninformed Search
- **Search and Models**

Search and Models

- Search operates over models of the world
 - The agent doesn't actually try all the plans out in the real world!
 - Planning is all “in simulation”
 - Your search is only as good as your models...

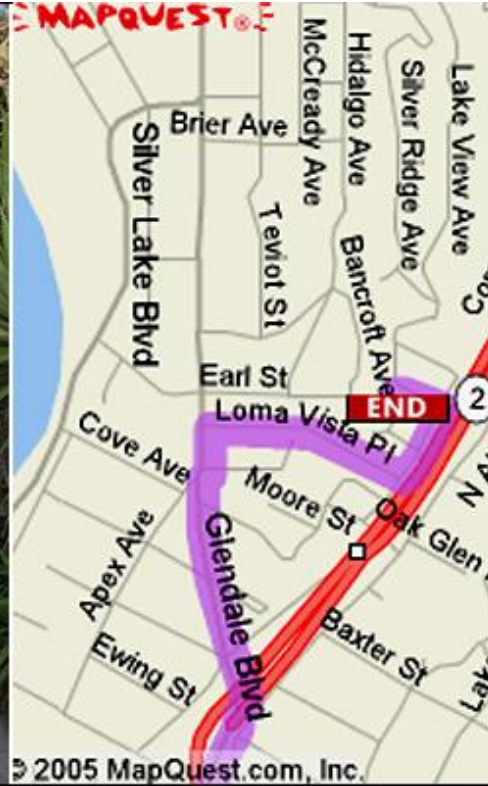


Uninformed Search



- Uninformed Search
- Search and Models
 - Search Gone Wrong

Search Gone Wrong



Uninformed Search



- Uninformed Search
- **Search and Models**
 - Search Gone Wrong

Search Gone Wrong



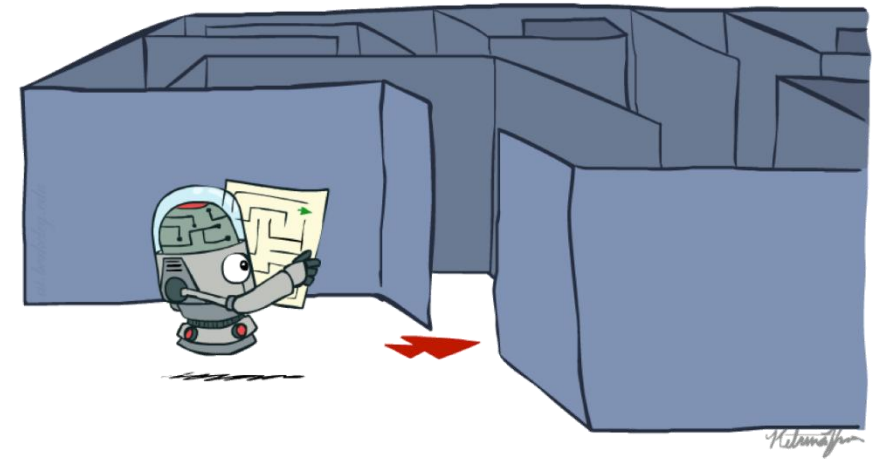
Uninformed Search



- Uninformed Search
- Search and Models
- **Recap**

Recap

- **Search problem:**
 - States (configurations of the world)
 - Actions and costs
 - Successor function (world dynamics)
 - Start state and goal test
- **Search tree:**
 - Nodes: represent plans for reaching states
 - Plans have costs (sum of action costs)
- **Search algorithm:**
 - Systematically builds a search tree
 - Chooses an ordering of the fringe (unexplored nodes)
 - Optimal: finds least-cost plans

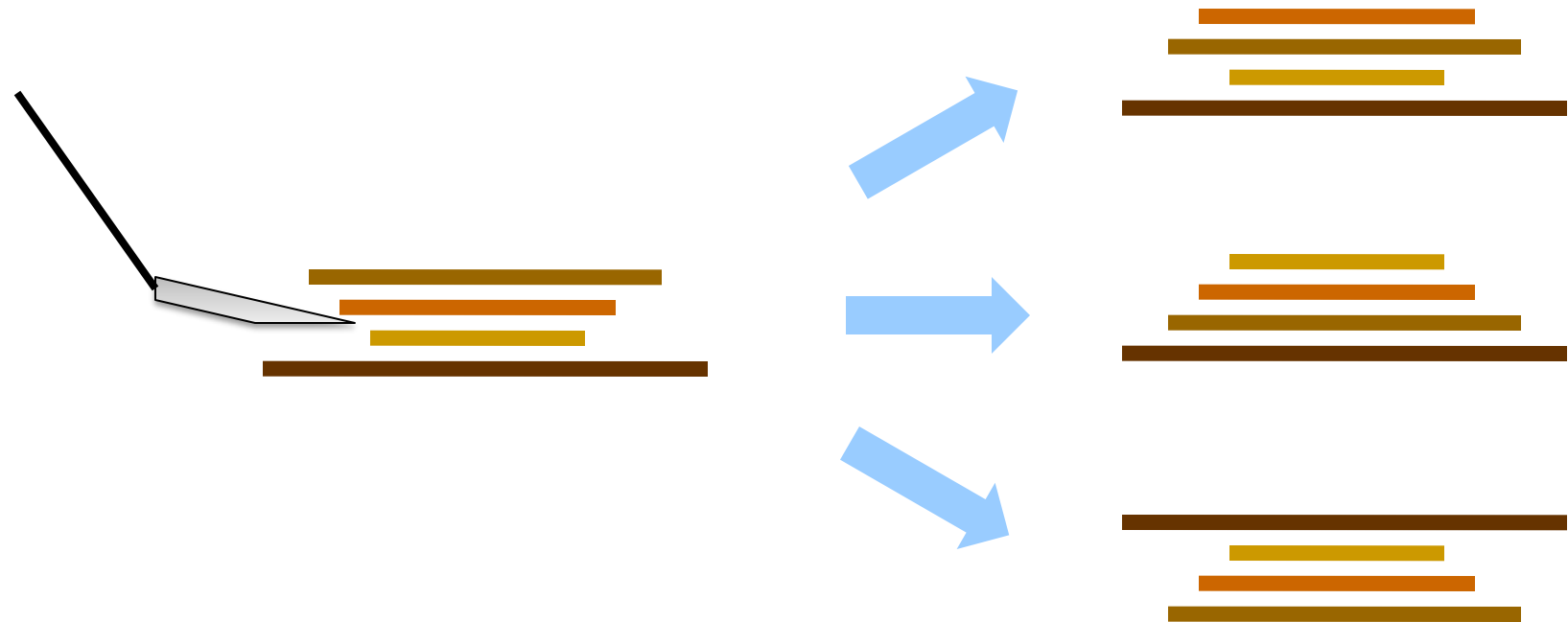


Uninformed Search



- Uninformed Search
- Search and Models
- **Recap**

Example: Pancake Problem



Cost: Number of pancakes flipped



Uninformed Search



- Uninformed Search
- Search and Models
- **Recap**

Example: Pancake Problem

BOUNDS FOR SORTING BY PREFIX REVERSAL

William H. GATES

Microsoft, Albuquerque, New Mexico

Christos H. PAPADIMITRIOU*†

Department of Electrical Engineering, University of California, Berkeley, CA 94720, U.S.A.

Received 18 January 1978

Revised 28 August 1978

For a permutation σ of the integers from 1 to n , let $f(\sigma)$ be the smallest number of prefix reversals that will transform σ to the identity permutation, and let $f(n)$ be the largest such $f(\sigma)$ for all σ in (the symmetric group) S_n . We show that $f(n) \leq (5n+5)/3$, and that $f(n) \geq 17n/16$ for n a multiple of 16. If, furthermore, each integer is required to participate in an even number of reversed prefixes, the corresponding function $g(n)$ is shown to obey $3n/2 - 1 \leq g(n) \leq 2n + 3$.



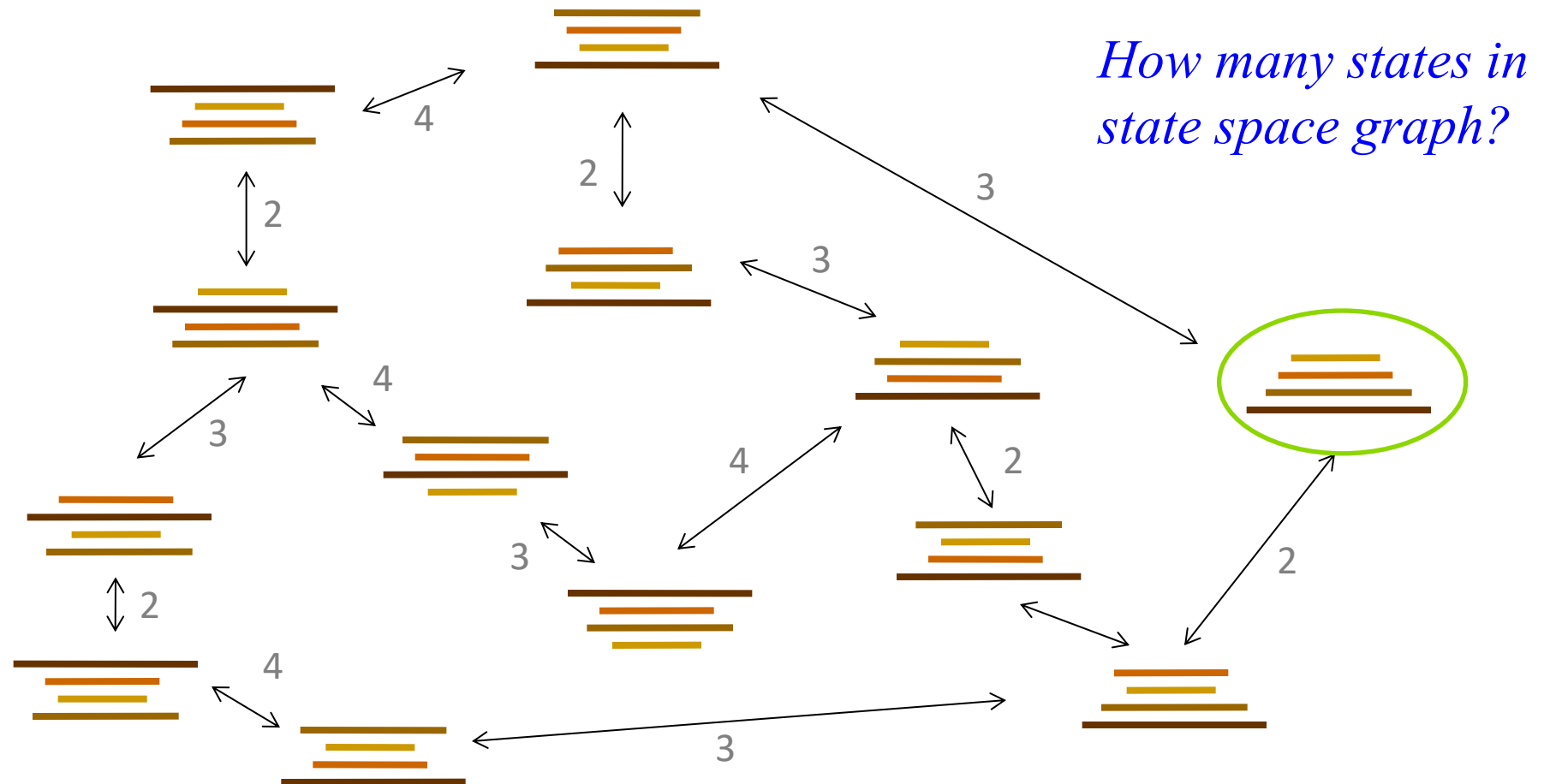
Uninformed Search



- Uninformed Search
- Search and Models
- **Recap**

Example: Pancake Problem

State space graph with costs as weights.



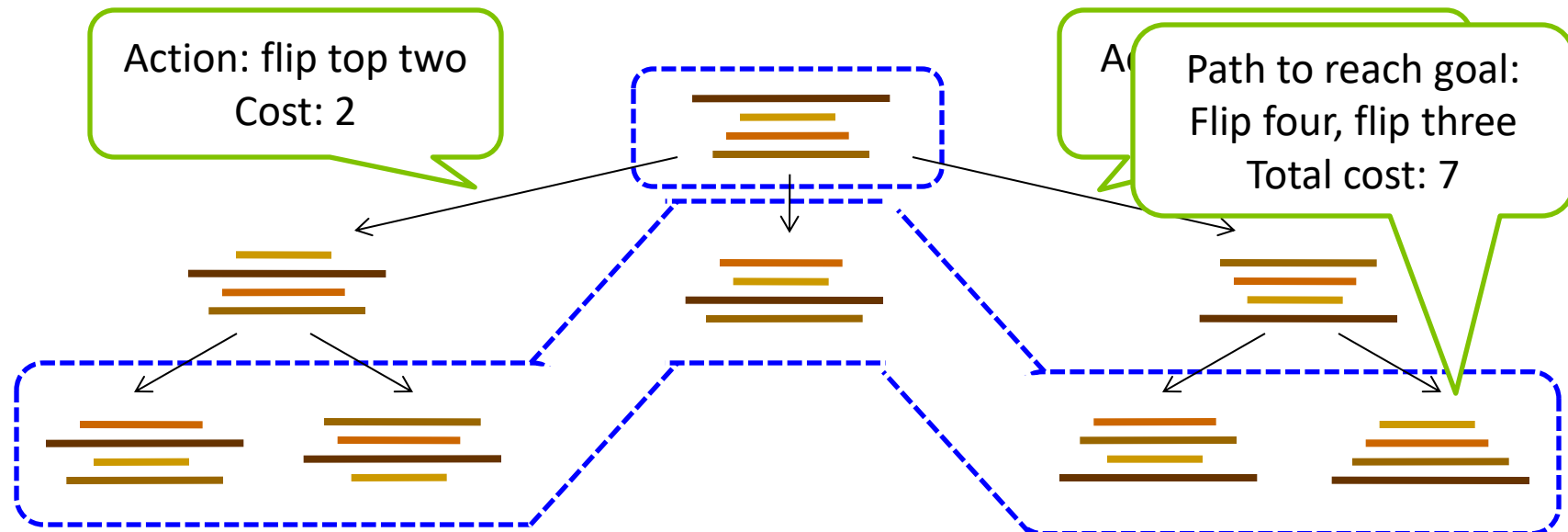
Uninformed Search



- Uninformed Search
- Search and Models
- **Recap**

General Tree Search

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```



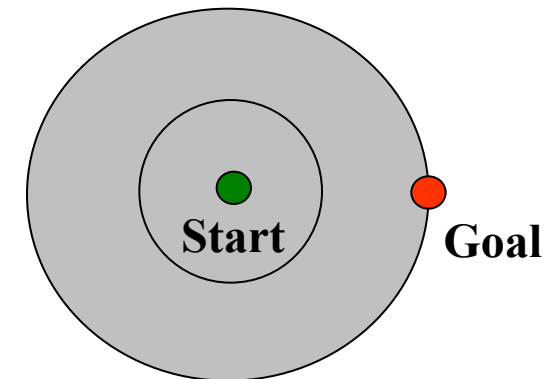
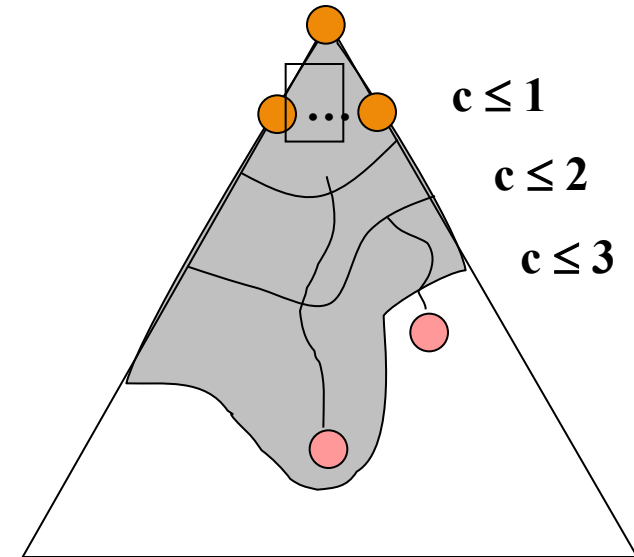
Informed Search



- Uninformed Search
- Search and Models
- **Recap**

Recap

- Strategy: expand lowest path cost
- The good: UCS is complete and optimal!
- The bad:
 - Explores options in every “direction”
 - No information about goal location
 - Both time and space complexity are exponential in effective depth

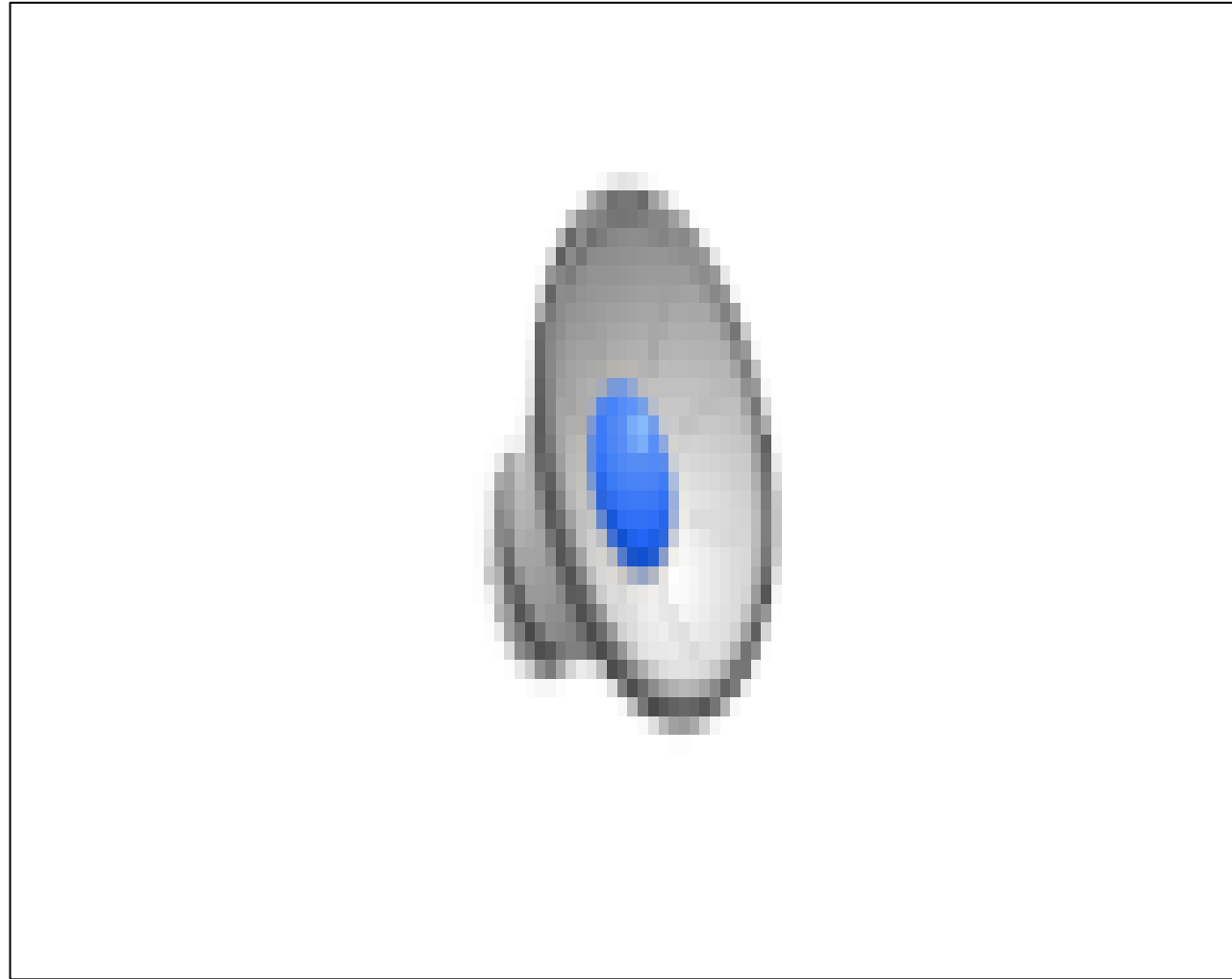


Informed Search



- Uninformed Search
- Search and Models
- **Recap**

Video of Demo Contour – UCS Empty

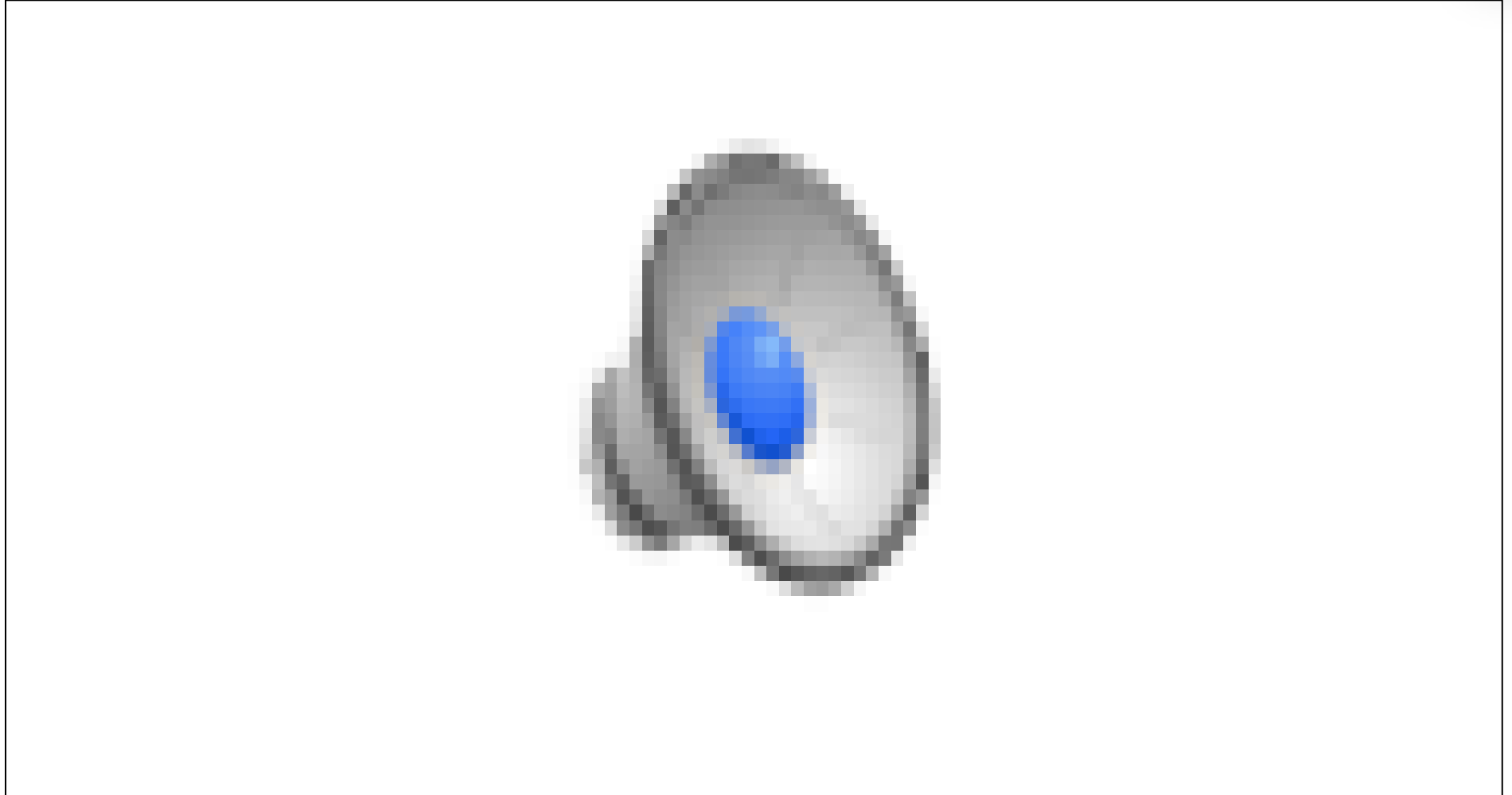


Informed Search



- Uninformed Search
- Search and Models
- **Recap**

Video of Demo Contour – UCS Pacman Small Maze

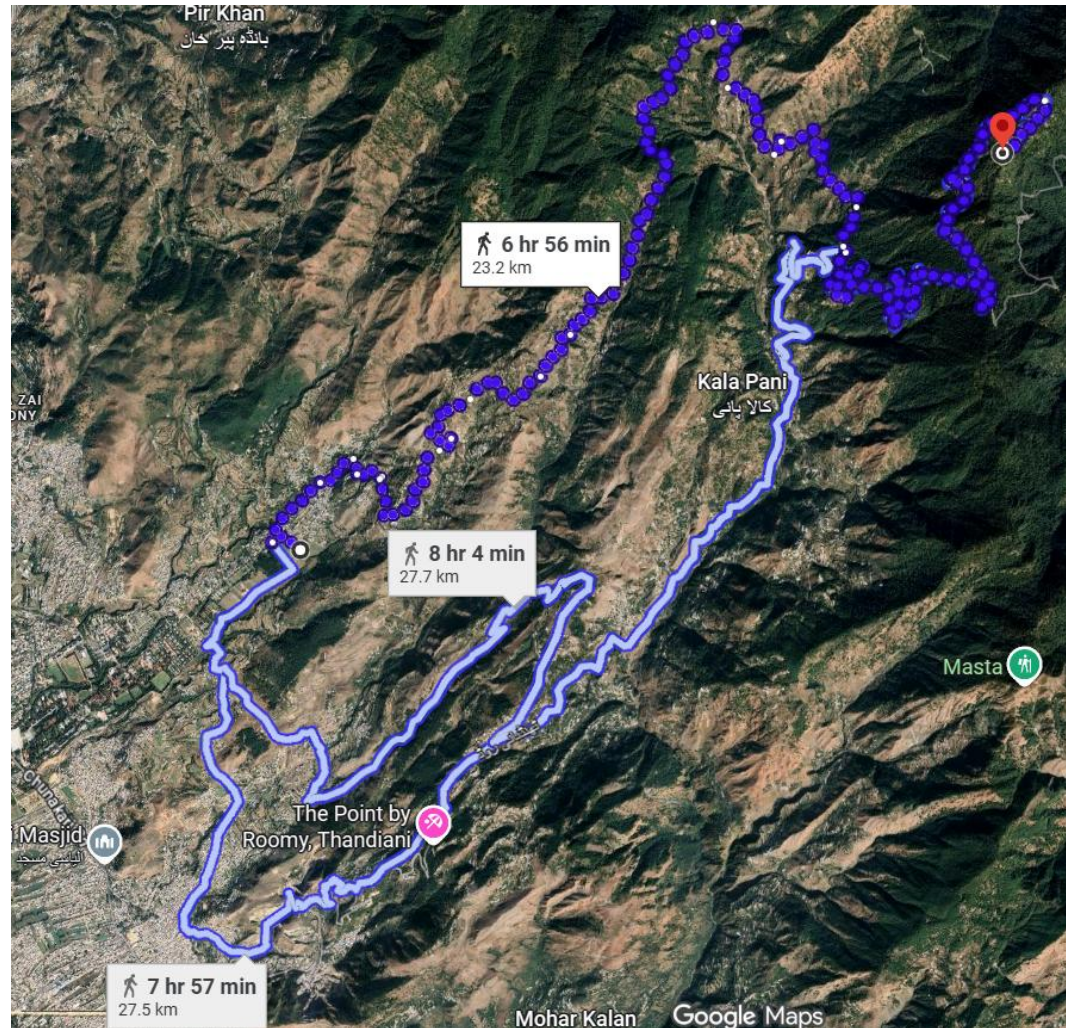


Informed Search



- Uninformed Search
- Search and Models
- **Informed Search**
 - Contrasting Examples

Contrasting Examples

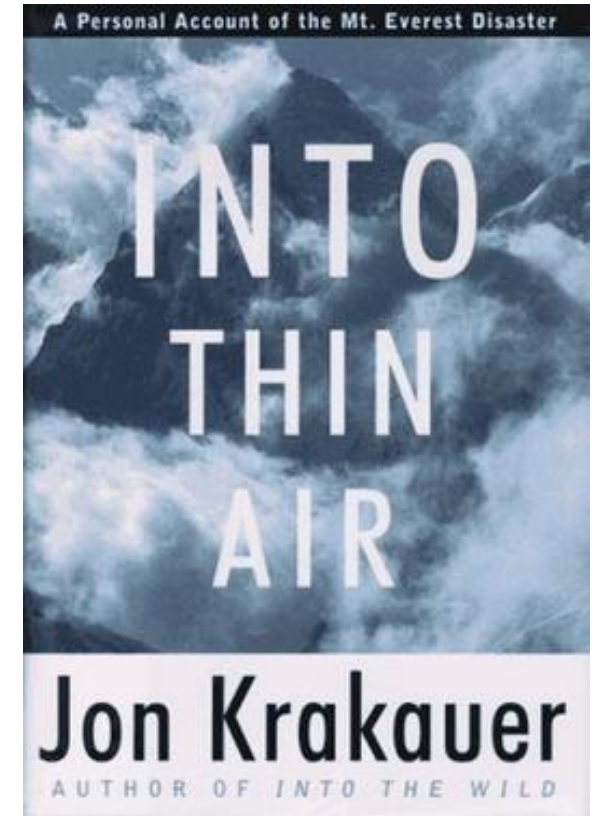


Informed Search



- Uninformed Search
- Search and Models
- **Informed Search**
 - Contrasting Examples

Contrasting Examples



Informed Search



- Uninformed Search
- Search and Models
- **Informed Search**
 - **Contrasting Examples**

Contrasting Examples

- The basic lesson of this lecture: it is MUCH easier to search when you have some idea where you're going!
 - (Subtle difference: these two examples are even harder than those we consider in lecture – in the woods, you actually have to wander around to conduct search, you can't just make plans computationally)



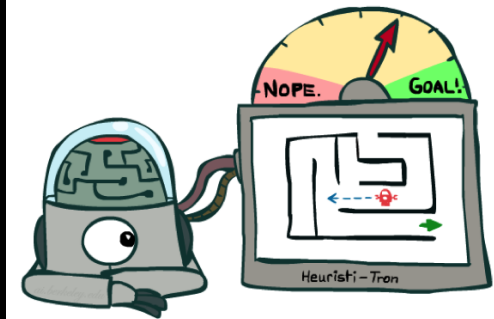
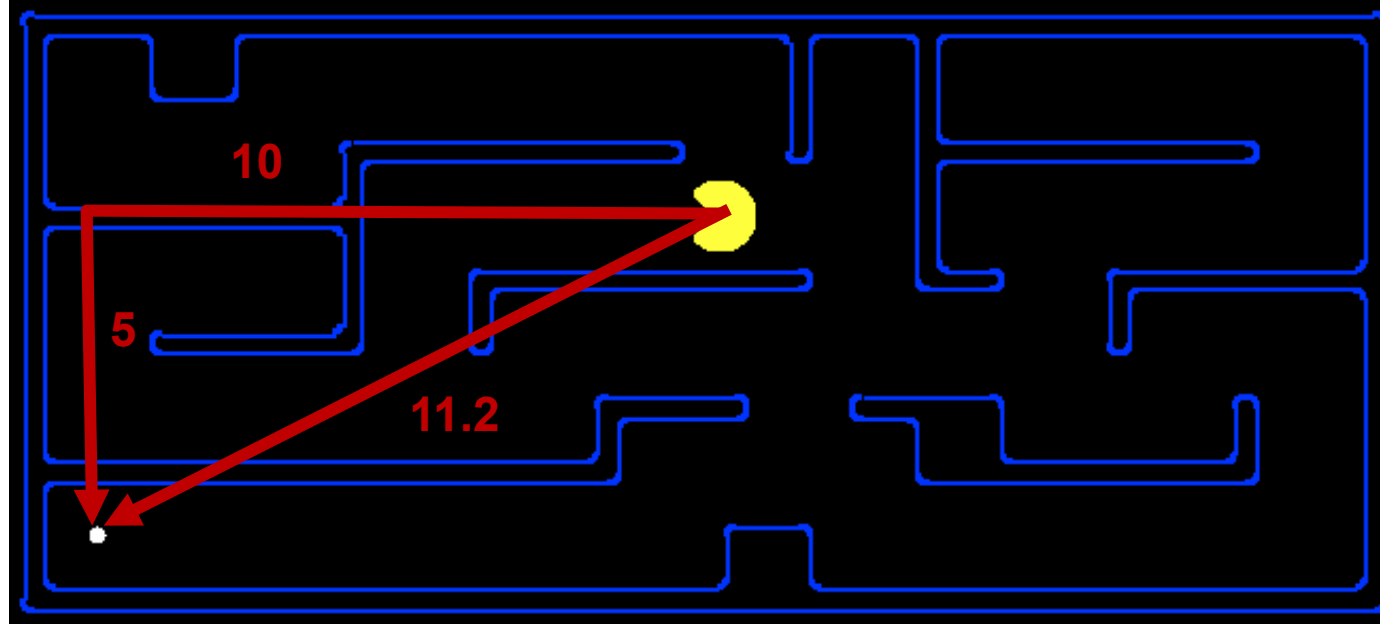
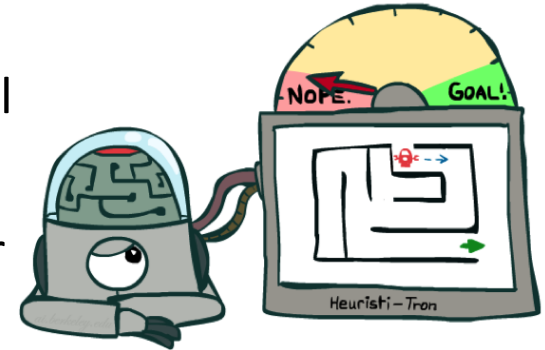
Informed Search



- Uninformed Search
- Search and Models
- **Informed Search**
 - Contrasting Examples
 - Search Heuristics

Search Heuristics

- A heuristic is:
 - A function that *estimates* how close a state is to a goal
 - Designed for a particular search problem
 - Examples: Manhattan distance, Euclidean distance for pathing

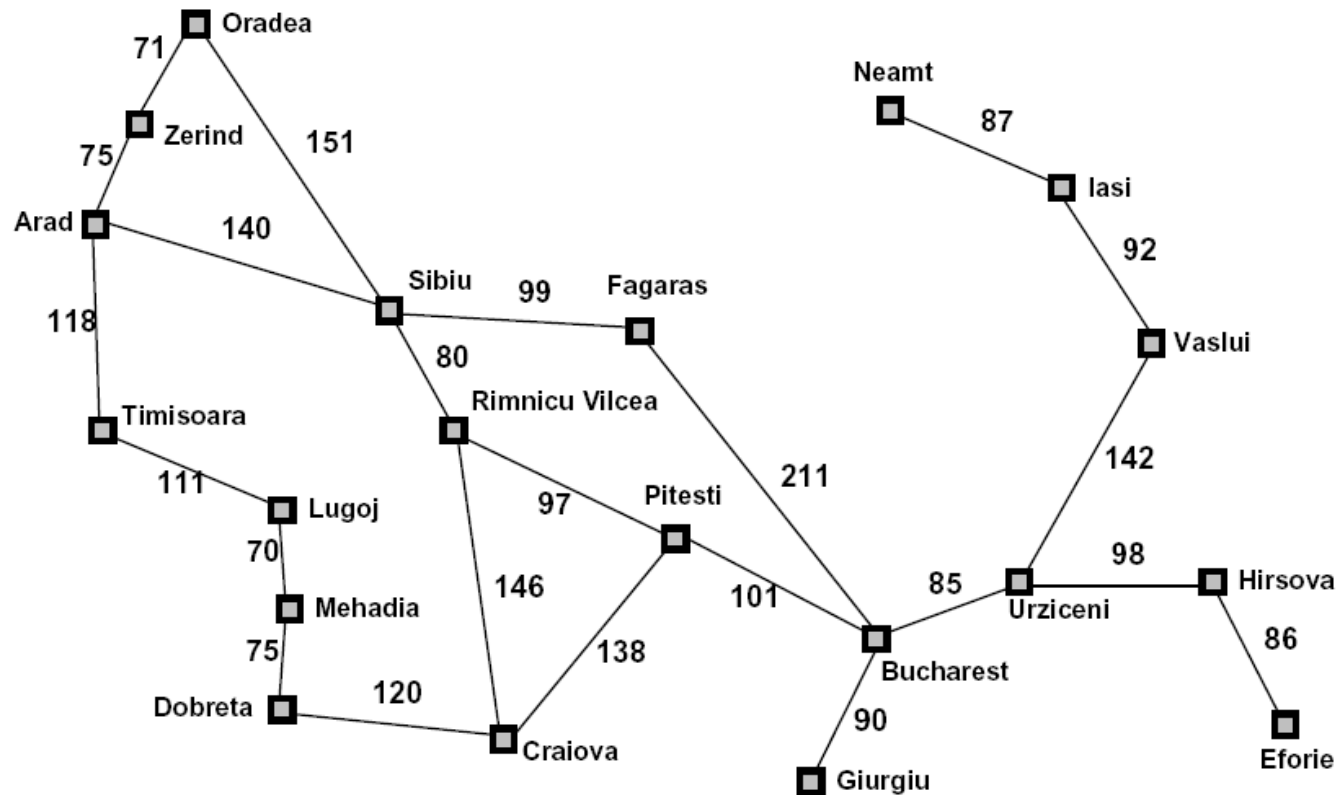


Informed Search



- Uninformed Search
- Search and Models
- **Informed Search**
 - Contrasting Examples
 - **Search Heuristics**

Heuristic Function Example



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

$h(x)$



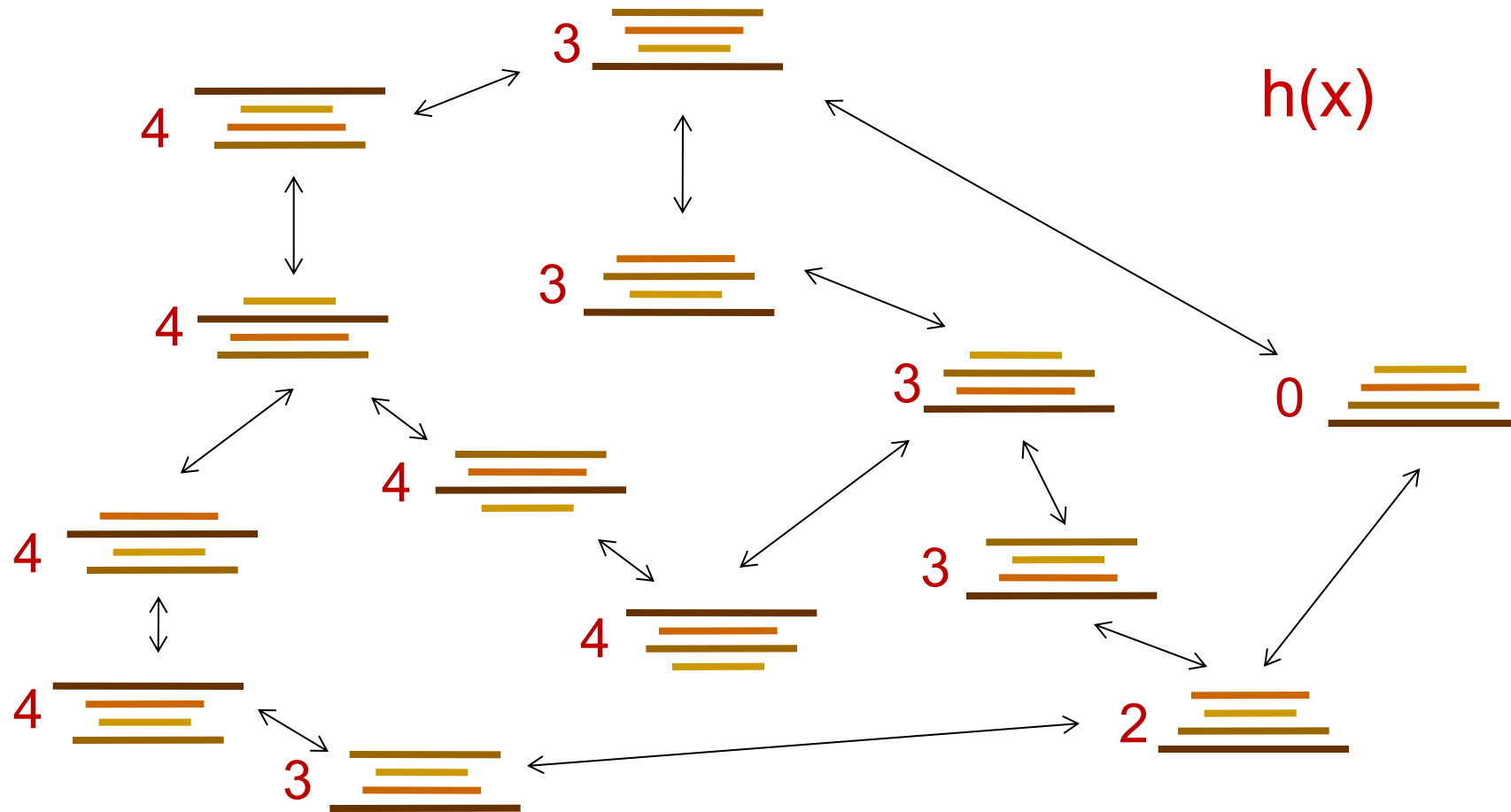
Informed Search



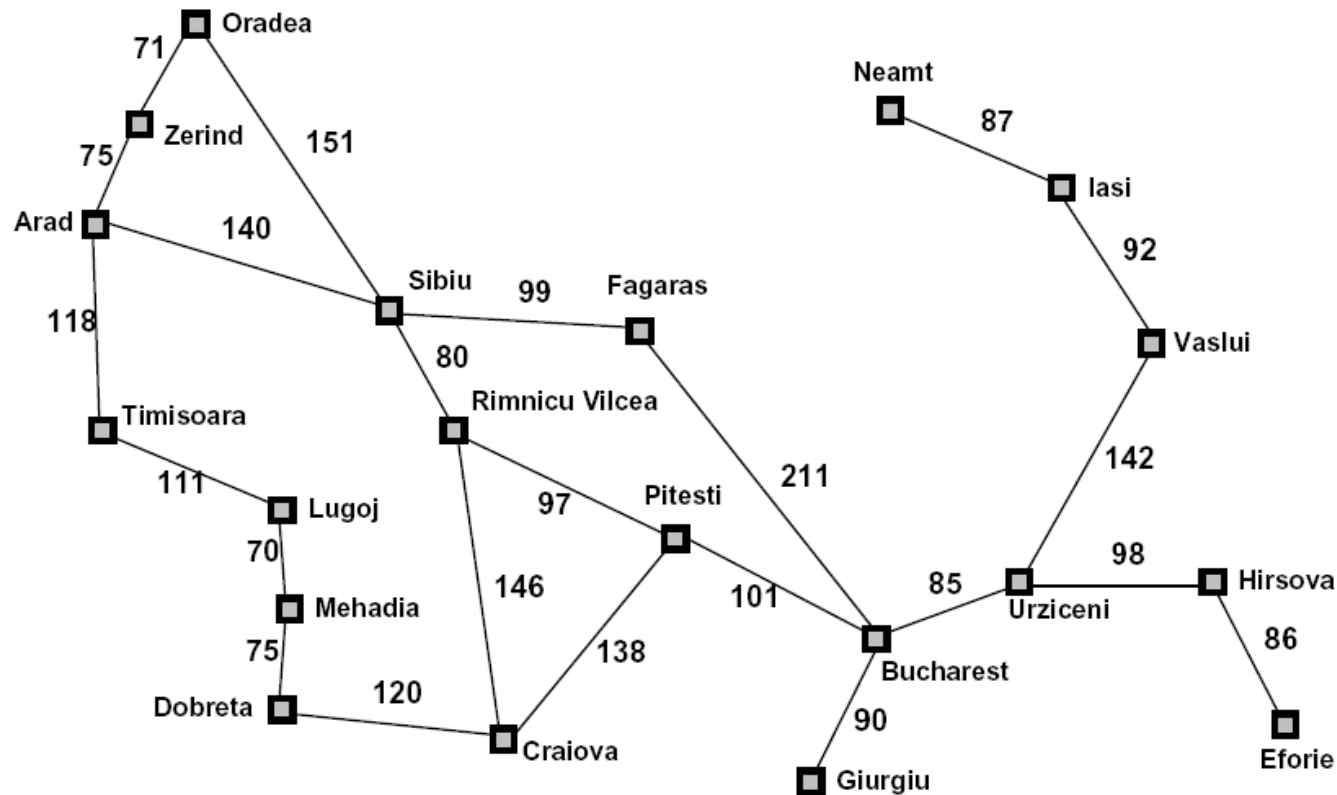
- Uninformed Search
- Search and Models
- **Informed Search**
 - Contrasting Examples
 - Search Heuristics

Heuristic Function Example

- Heuristic: the ID number of the largest pancake that is still out of place



- ## Heuristic Function Example – Greedy Search


$$h(\mathbf{x})$$

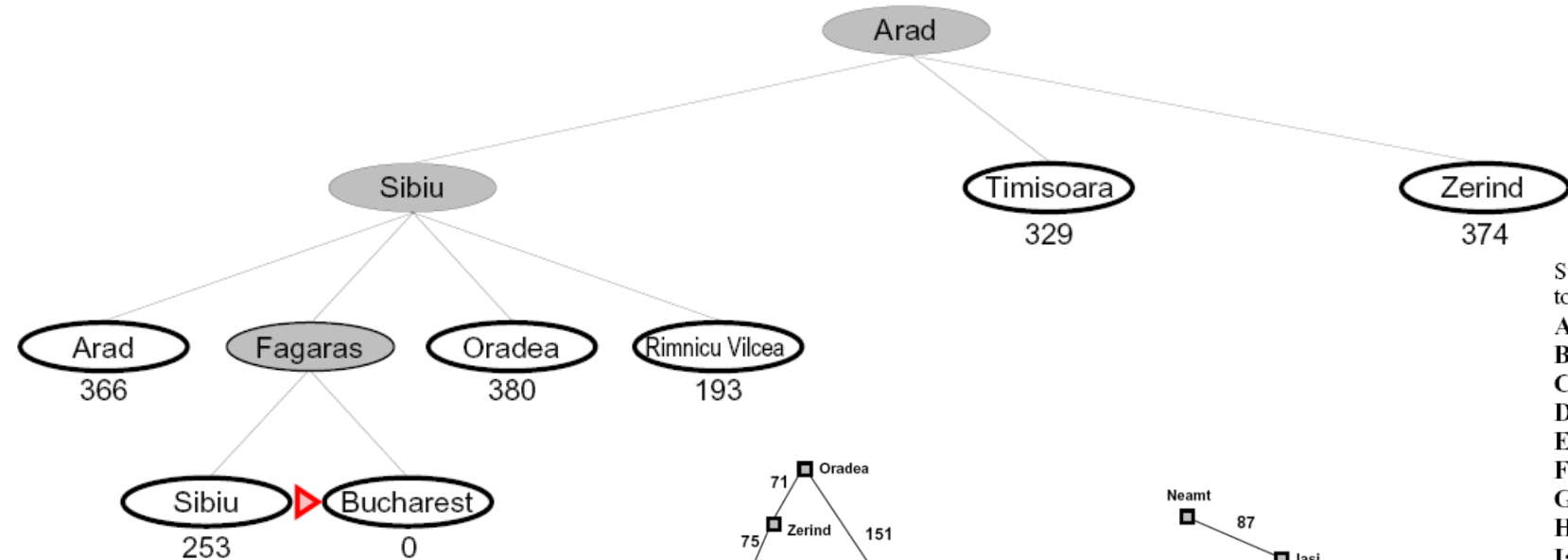

Informed Search



- Uninformed Search
- Search and Models
- **Informed Search**
 - Contrasting Examples
 - Search Heuristics
 - **Greedy Search**

Heuristic Function Example – Greedy Search

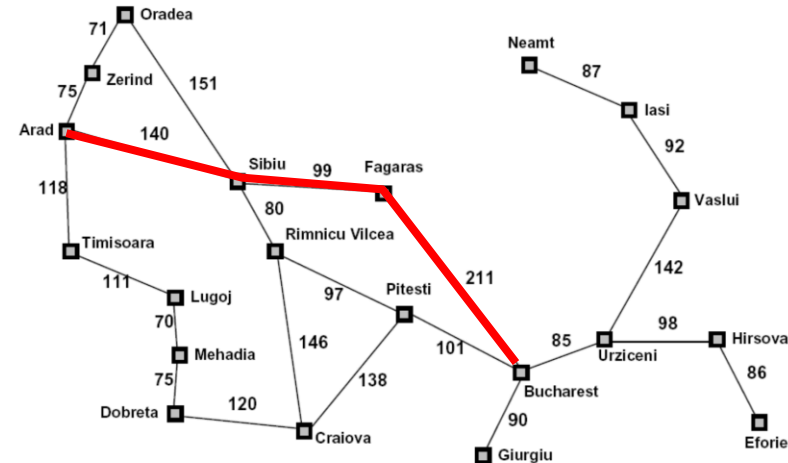
- Expand the node that seems closest...



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

- What can go wrong?



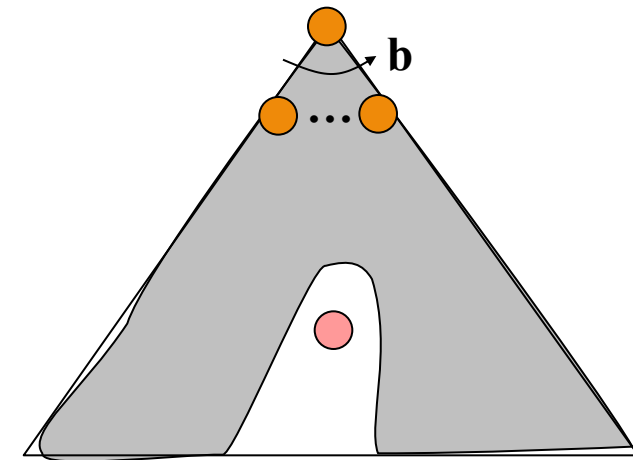
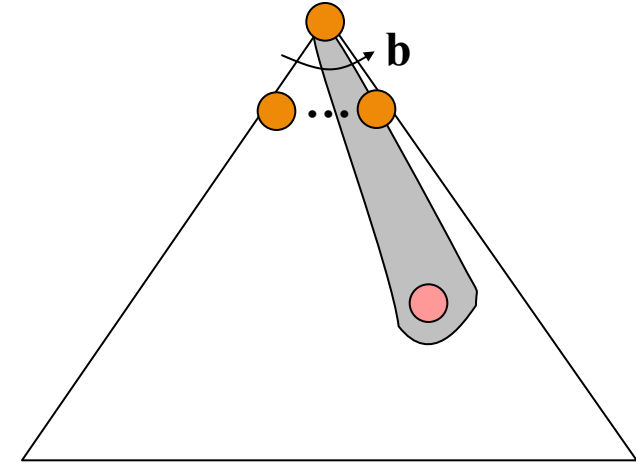
Informed Search



- Uninformed Search
- Search and Models
- **Informed Search**
 - Contrasting Examples
 - Search Heuristics
 - **Greedy Search**

Greedy Search

- Strategy: expand a node that you think is closest to a goal state
 - Heuristic: estimate of distance to nearest goal for each state
- A common case:
 - Best-first takes you straight to the (wrong) goal
- Worst-case: like a badly-guided DFS

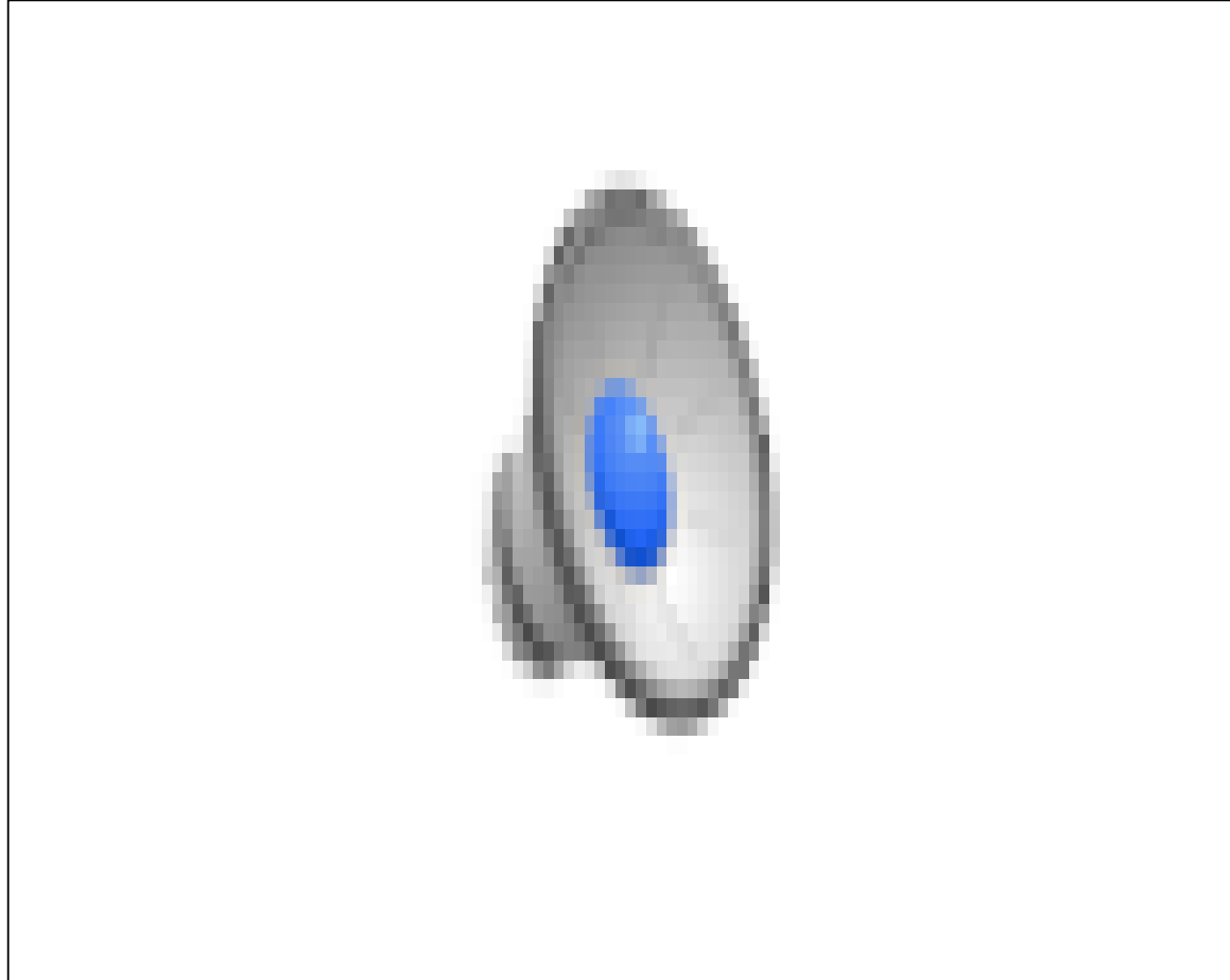


Informed Search



- Uninformed Search
- Search and Models
- **Informed Search**
 - Contrasting Examples
 - Search Heuristics
 - **Greedy Search**

Greedy Search – Video of Demo Contours (Empty)



Informed Search



- Uninformed Search
- Search and Models
- **Informed Search**
 - Contrasting Examples
 - Search Heuristics
 - **Greedy Search**

Greedy Search – Video of Demo Contours (Pacman Small Maze)

